

Dokumentacja Techniczna Projektu: Metaheurystyczny Algorytm Artemisinin Optimizer (AO) w Problemie Kwadratowego Zagadnienia Przydziału (QAP)

Autorzy:

Błażej Kwaśny, Patryk Lorenc

9 czerwca 2026

Streszczenie

Niniejszy dokument stanowi kompletną dokumentację techniczną implementacji hybrydowych algorytmów optymalizacyjnych inspirowanych naturą, opartych na procesie ekstrakcji artemizyny (*Artemisinin Optimizer*, AO). Algorytm został zaadaptowany oraz zaimplementowany w celu rozwiązywania trudnych kombinatorycznych problemów optymalizacji dyskretnej, na przykładzie Kwadratowego Zagadnienia Przydziału (QAP). Dokumentacja obejmuje opis matematyczny transformacji przestrzeni, specyfikację trzech wariantów algorytmu (klasycznego AO, WRAO z rankingiem ważonym oraz AO-PMX z iniekcją struktur elitarnych) oraz szczegółową analizę programistyczną modułów systemu.

Spis treści

1	Wprowadzenie i geneza algorytmu	3
1.1	Ogólna idea i źródło inspiracji	3
1.2	Kwadratowe Zagadnienie Przydziału (QAP)	3
2	Reprezentacja przestrzeni i mapowanie (Ciągła vs Dyskretna)	3
2.1	Transformacja ROV (Rank Order Value)	4
2.2	Mapowanie wsteczne (Reverse Mapping)	4
3	Opis modułów i funkcji pomocniczych (Numba Utilities)	5
3.1	rov_mapping_numba(continuous_vector)	5
3.2	calculate_qap_fitness_numba(A, B, permutation)	5
3.3	map_back_numba(permutation)	5
3.4	hamming_distance_numba(p1, p2)	5
3.5	elite_injection_numba(target_perm, elite_perm, injection_size)	6
3.6	full_2opt_numba(permutation, A, B, current_f, max_f)	6
4	Charakterystyka trzech wersji algorytmu	7
4.1	Klasyczny Artemisinin Optimizer (Wersja AO)	7
4.2	Wersja z rankingiem ważonym (Wersja WRAO)	7
4.3	Połączenie AO z mutacją PMX (Wersja PMX)	7
5	Architektura systemu i integracja programistyczna	9
5.1	data_loader.py – Ładowanie instancji testowych	9
5.2	benchmark.py – Moduł ewaluacyjny	9
5.3	main.py – Główny skrypt uruchomieniowy	9

1 Wprowadzenie i geneza algorytmu

1.1 Ogólna idea i źródło inspiracji

Algorytm *Artemisinin Optimizer* (AO) to nowoczesna metaheurystyka populacyjna zainspirowana fizykochemicznymi procesami zachodzącymi podczas przemysłowej ekstrakcji oraz oczyszczania artemizyny – organicznego związku chemicznego stosowanego jako kluczowy lek w terapii malarii.

Wzorzec biologiczno-chemiczny opiera się na fazach podziału, dyfuzji, eliminacji zanieczyszczeń oraz ruchu cząsteczek w układzie wielofazowym (ciecz-ciecz lub ciecz-ciało stałe). W kontekście algorytmicznym proces ten jest modelowany jako ciągła eksploracja przestrzeni poszukiwań przez agentów populacji, gdzie globalna zbieżność realizowana jest poprzez dwa główne zjawiska:

1. **Faza eliminacji / Shaking (Perturbacja):** Odpowiednik losowych zderzeń cząsteczek i usuwania frakcji balastowych. Wstrzykuje losowy szum gaussowski w celu ucieczki z minimów lokalnych.
2. **Ruch w kierunku lidera / Trajektoria AO:** Przepływ masowy i krystalizacja substancji aktywnej, modelowana jako deterministyczno-stochastyczne zbieganie agentów w stronę najlepiej przystosowanego rozwiązania.

1.2 Kwadratowe Zagadnienie Przydziału (QAP)

Problem QAP (*Quadratic Assignment Problem*) polega na optymalnym przypisaniu n obiektów do n lokalizacji przy danych macierzach przepływu A oraz odległości B . Formalnie dąży się do znalezienia permutacji $\pi \in S_n$ minimalizującej funkcję kosztu (przyastosowania) zdefiniowaną jako:

$$f(\pi) = \sum_{i=1}^n \sum_{j=1}^n A_{i,j} \cdot B_{\pi(i),\pi(j)} \quad (1)$$

2 Reprezentacja przestrzeni i mapowanie (Ciągła vs Dyskretna)

Klasyczny algorytm AO operuje w ciągłej przestrzeni matematycznej $\mathbb{R}^D$, co pozwala na płynne modelowanie fizycznych przesunięć i dyfuzji. Jednakże, problem QAP z natury wymaga reprezentacji dyskretnej (permutacyjnej). W tym celu zaimplementowano dwukierunkowy pomost matematyczny między oboma światami.

Tabela 1: Porównanie operowania na przestrzeniach poszukiwań

Operacja / Faza algorytmu	Przestrzeń	Typ danych
Inicjalizacja i ruch agentów (AO)	Ciągła $\mathbb{R}^D \in [-1, 1]^D$	<code>numpy.float64</code>
Mapowanie pozycji na permutację	Transformacja (ROV)	$\mathbb{R}^D \rightarrow \mathbb{N}^D$
Ocena fitness, Local Search (2-opt), PMX	Dyskretna (Permutacje)	<code>numpy.int64</code>
Aktualizacja pozycji po optymalizacji	Mapowanie wsteczne	$\mathbb{N}^D \rightarrow \mathbb{R}^D$

2.1 Transformacja ROV (Rank Order Value)

Przejście z wektora ciągłego $X = [x_1, x_2, \dots, x_D]$ do permutacji dyskretnej π realizowane jest za pomocą techniki *Rank Order Value*. Elementy wektora ciągłego są sortowane rosnąco, a permutację stanowią oryginalne indeksy pozycji tych elementów przed sortowaniem. Można to zapisać operacją:

$$\pi = \text{argsort}(X) \quad (2)$$

2.2 Mapowanie wsteczne (Reverse Mapping)

Aby algorytm ciągły mógł dalej ewoluować na bazie wyników uzyskanych przez lokalny optymalizator dyskretny (np. 2-opt), zmodyfikowana permutacja musi zostać przekształcona z powrotem na wektor ciągły w dziedzinie $[-1, 1]^D$. Dla zadanej pozycji w permutacji o randze $r \in [0, D - 1]$, wartość ciągła x_i jest wyznaczana jako:

$$x_i = \left(\frac{r}{D - 1} \right) \cdot 2.0 - 1.0 \quad (3)$$

3 Opis modułów i funkcji pomocniczych (Numba Utilities)

Wszystkie krytyczne obliczeniowo funkcje pomocnicze zostały uniezależnione od obiektów wysokiego poziomu i przyspieszone za pomocą kompilatora JIT (`numba.njit(cache=True)`), eliminując narzut interpretera Pythona.

3.1 `rov_mapping_numba(continuous_vector)`

- **Wejście:** Jednowymiarowy wektor ciągły typu `float64`.
- **Wyjście:** Wektor indeksów typu `int64` reprezentujący permutację (0-indexed).
- **Działanie:** Wywołuje niskopoziomową funkcję `np.argsort()`. Sortuje wartości wektora ciągłego i zwraca tablicę indeksów, które porządkują ten wektor.

3.2 `calculate_qap_fitness_numba(A, B, permutation)`

- **Wejście:** Macierz przepływu A , macierz odległości B (obie jako ciągłe struktury `int64` w pamięci), wektor permutacji.
- **Wyjście:** Skalar typu `int64` będący całkowitym kosztem dopasowania.
- **Działanie:** Oblicza iloczyn Frobeniusa dla zadanej permutacji. Realizuje to za pomocą zagnieżdżonej, podwójnej pętli iterującej po indeksach od 0 do $n - 1$:

```
1 total += A[i, j] * B[permutation[i], permutation[j]]  
2
```

3.3 `map_back_numba(permutation)`

- **Wejście:** Wektor permutacji dyskretnej (`int64`).
- **Wyjście:** Zmapowany wektor ciągły (`float64`) o wymiarowości D .
- **Działanie:** Najpierw rekonstruuje rangi obiektów poprzez odwrócenie tablicy permutacji, a następnie normalizuje je liniowo do przedziału $[-1, 1]$.

3.4 `hamming_distance_numba(p1, p2)`

- **Wejście:** Dwie permutacje $p1$ oraz $p2$ o tej samej długości.
- **Wyjście:** Skalar typu `int64` reprezentujący odległość Hamminga.
- **Działanie:** Wyznacza liczbę pozycji, na których elementy obu permutacji różnią się od siebie. Wykorzystywana do wyznaczania intensywności mutacji krzyżowej w module PMX.

3.5 elite_injection_numba(target_perm, elite_perm, injection_size)

- **Wejście:** Permutacja bazowa (`target_perm`), permutacja lidera (`elite_perm`), rozmiar wstrzykiwanego bloku (`injection_size`).
- **Wyjście:** Nowa, poprawna permutacja potomna.
- **Działanie:** Operator typu PMX/OX. W pierwszym kroku kopiuje fragment o długości `injection_size` bezpośrednio z pozycji początkowych z wektora `elite_perm` do potomka. W drugim kroku uzupełnia pozostałe puste miejsca genami z `target_perm`, zachowując ich oryginalną kolejność i bezwzględnie unikając duplikacji genów (zapewnienie komparatywnej poprawności przydziału).

3.6 full_2opt_numba(permutation, A, B, current_f, max_f)

- **Wejście:** Permutacja, macierze A i B , licznik ewaluacji (`current_f`) oraz limit budżetu (`max_f`).
- **Wyjście:** Trzyelementowa krotka: (`best_permutation`, `best_cost`, `updated_f`).
- **Działanie:** Algorytm przeszukiwania lokalnego (*Local Search*). Systematycznie przegląda całe otoczenie rozwiązania wykonując zamiany par genów (*pairwise swaps*) typu 2-opt. Jeśli zamiana przynosi poprawę funkcji celu, zostaje ona utrwalona, a proces przeszukiwania otoczenia startuje od nowa. Działa do momentu osiągnięcia lokalnego optimum (brak poprawy w pełnym przebiegu) lub wyczerpania budżetu `max_f`.

4 Charakterystyka trzech wersji algorytmu

Algorytm sterujący współgra z parametrami nieliniowymi zależnymi od postępu obliczeń $\text{progress} = \frac{f}{\text{MaxF}}$:

- Parametr perturbacji: $K = 0.4 \cdot (1 - \text{progress})$ (maleje liniowo, stabilizując populację).
- Współczynnik trajektorii: $c = 2.0 \cdot \exp(-\text{progress}^2)$ (determinuje siłę przyciągania do lidera).

4.1 Klasyczny Artemisinin Optimizer (Wersja AO)

Klasyczny wariant (`ArtemisininOptimizer`) opiera ewolucję każdego agenta na pojedynczym, globalnym liderze układu (`best_agent`):

- **Mechanizm Shaking:** Jeśli wylosowana wartość stochastyczna jest mniejsza od K , element wektora ciągłego agenta mutuje według wzoru:

$$X_{i,j}^{\text{new}} = \text{best_agent}_j + \mathcal{N}(0, 0.4) \quad (4)$$

- **Mechanizm Ruchu:** W przeciwnym wypadku agent zbliża się do lidera:

$$X_{i,j}^{\text{new}} = c \cdot \text{best_agent}_j + (1 - c) \cdot X_{i,j} \quad (5)$$

4.2 Wersja z rankingiem ważonym (Wersja WRAO)

Wariant `WeightedArtemisininOptimizer` rezygnuje z podążania za jednym punktem na rzecz tzw. *Weighted Leader Vector*, obliczanego w metodzie `_calculate_weighted_leader()`:

1. Populacja jest sortowana według przystosowania.
2. Wybierana jest elita najlepszych agentów w liczbie określonej przez ułamek `ranking_portion`.
3. Przypisuje się im wagi odwrotnie proporcjonalne do wartości funkcji celu: $w_i = \frac{1}{f(\pi_i) + \varepsilon}$.
4. Nowym sztucznym liderem staje się ważona średnia pozycji elity w przestrzeni ciągłej:

$$X^{\text{weighted_leader}} = \sum_{i \in \text{TopK}} w_i \cdot X_i \quad (6)$$

Wszystkie operacje ruchu i mutacji w pętli głównej wykonuje się względem obliczonego $X^{\text{weighted_leader}}$.

4.3 Połączenie AO z mutacją PMX (Wersja PMX)

Wariant `PMXOptimizer` wprowadza bezpośredni operator rekombinacji genetycznej do struktur dyskretnych. Co określoną liczbę iteracji (`injection_period`), wyznaczana jest odległość Hamminga pomiędzy wygenerowaną pozycją ciągłą agenta (po konwersji ROV), a najlepszą globalną permutacją sekwencyjną (`best_perm`).

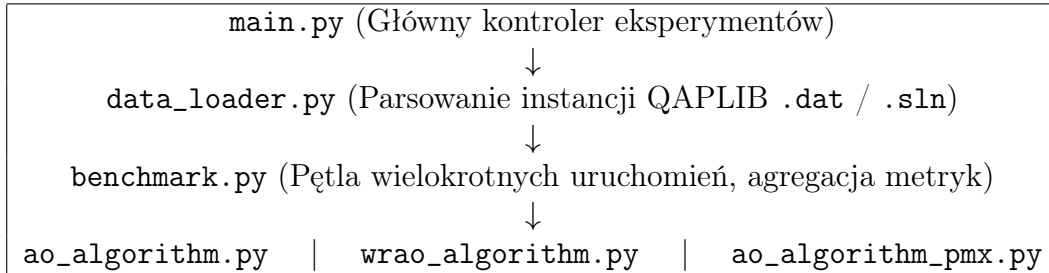
Rozmiar iniekcji strukturalnej definiuje równanie:

$$\text{injection_size} = \max(1, \lfloor \text{hamming_dist} \cdot \text{injection_rate} \rfloor) \quad (7)$$

Poprzez funkcję `elite_injection_numba` cechy topologiczne globalnego optimum są bezpośrednio zaszczepiane w agencie przed uruchomieniem fazy *Local Search*, co gwałtownie przyspiesza przeszukiwanie przestrzeni w obszarach obiecujących.

5 Architektura systemu i integracja programistyczna

Projekt został podzielony na modularne skrypty realizujące zasady separacji odpowiedzialności:



Rysunek 1: Schemat przepływu danych i hierarchii modułów projektowych

5.1 data_loader.py – Ładowanie instancji testowych

Moduł odpowiedzialny za czytanie danych wejściowych w standardzie biblioteki QAPLIB.

- `load_instance(file_path)`: Odczytuje plik tekstowy `.dat`, pobiera wymiar problemu n , a następnie parsuje ciąg liczb tworząc dwie macierze dwuwymiarowe A oraz B o rozmiarach $n \times n$.
- `load_solution(file_path)`: Odczytuje pliki `.sln`, ekstrahując globalną wartość optymalną (`optimal`) oraz wektor wzorcowej permutacji przydziału.

5.2 benchmark.py – Moduł ewaluacyjny

Główna funkcja uruchomieniowa `benchmark()` standaryzuje proces testowy. Dla każdego zadanego algorytmu wykonuje niezależną sekwencję `n_runs` (np. 30 powtórzeń). Odpowiada za:

- Izolację czasu wykonania za pomocą `time.perf_counter()`.
- Wyznaczenie wskaźnika względnego błędu procentowego GAP względem znanej referencji optymalnej:

$$\text{GAP} = \frac{\text{Best_Score} - \text{Optimum}}{\text{Optimum}} \cdot 100\% \quad (8)$$

- Zbieranie i ujednolicanie migawek populacji (*Population Snapshots*) w czterech krytycznych fazach: `start`, `mid_1` (33% MaxF), `mid_2` (66% MaxF) oraz `end`.

5.3 main.py – Główny skrypt uruchomieniowy

Realizuje pełną pętlę automatyzacji badań: skanuje foldery scenariuszy, parsuje pliki danych, powołuje instancje optymalizatorów z populacją o rozmiarze $N = 5000$ i limitem budżetu $\text{MaxF} = 2\,000\,000$ ewaluacji. Odpowiada za strukturyzację zapisów wyników w katalogu `Result/[Nazwa_Instancji]`, eksportując pliki statystyczne `summary_comparison.csv`, dane surowe `detailed_runs.csv`, raporty `final_report.txt` oraz kompletne logi zrzutów populacji w formacie 1-indexed do osobnych plików tekstowych.